

מזהה קורס: 20905 שם קורס: שפות תכנות

מספר שאלה	ציון מירבי	ציון שאלה סופי
1	35.00	30.00
2	40.00	35.00
3	25.00	25.00

ציון בחינה סופי : 90.00

הבחינה הבדוקה בעמודים הבאים

פתרון שאלה 1

השינויים לקובץ lang.scm הם כדלהלן:

שינוי כללי הגזירה הקיימים עבור הגדרת פרוצדורה והפעלת פרוצדורה:

```
(expression
  ("proc" "(" (separated-list identifier "=" expression ",") ")" expression)
  proc-exp)
```



```
(expression
  ("(" expression (separated-list identifier "=" expression ",") ")")
  call-exp)
```

כעת במפרש כלומר ב- interp.scm בתוך value-of יש לעדכן את חישוב הערך של המקרים המתאימים:

```
(proc-exp (ids defs body)
  (let ((defs1 (map (lambda (exp) (value-of exp env)) defs)) )
    (proc-val (procedure ids defs1 body env))))
```



כאן שינתי ב- data-structure.scm את ההגדרה של בנאי procedure לקבל פרמטרים שונים.

```
(define-datatype proc proc?
  (procedure
    (ids (list-of symbol?))
    (defs (list-of expval?))
    (body expression?)
    (env environment?)))
```



בנוסף ב- value-of שינוי ה- case של הפעלת פרוצדורה כדלהלן:

```
(call-exp (rator parms exps)
  (let ((proc (expval->proc (value-of rator env)))
        (args (map (lambda (exp) (value-of exp env)) exps)))
```



```
(apply-procedure proc params args)))
```

ומקוד זה קל להסיק ששיניתי גם את הפרוצדורה apply-procedure כך:

```
(define apply-procedure
  (lambda (proc1 params args)
    (cases proc proc1
      (procedure (ids defs body saved-env)
        (value-of body (extend-multi ids defs params args saved-env))))))
```

פרוצדורה עזר extend-multi:

```
(define (extend-multi ids defs params args env)
  (if (null? ids) env
      (let (
            (index (index-of (car ids) params)))
              (if (= index -1) (extend-multi (cdr ids) (cdr defs) params args (extend-env (car ids) (car defs)
env))
              ;ELSE
              (extend-multi (cdr ids) (cdr defs) params args (extend-env (car ids) (list-ref args index) env))
            )
        )
```

חסר טיפול במקרה שparams מכיל משתנה בשם שאינו אחד מהפרמטרים של הפרוצדורה

(הערה: אין צורך לוודא שאורכי הרשימות args ו-params שוות אורך כי כללי הגזירה מבטיחים זאת)

הסבר במילים: מקבלים רשימת שמות משתנים בהגדרה של הפרוצדורה (ids), רשימה ערכי ברירת מחדל (defs), רשימת משתנים שניתן להם ערך בזימון (params) ורשימת ערכים. עבור כל משתנה בהגדרה, אם קיים ברשימת המשתנים params אז לוקחים את הערך מ-args במקום המתאים.

הפרוצדורה index-of לא תובא כאן מקוצר זמן אבל זו שאלה קלה והפרדיקט equal? עם קידום counter עד שהרשימה ריקה או שנמצא הערך פותר בקלות את הבעיה.

בנוסף, ייתכן ואיני זוכר את סדר הארגומנטים לפרוצדורות קיימות כגון list-ref (קודם הרשימה או האינדקס?) אך זהו עניין שולי.

לסיכום, בכל קריאה רקורסיבית / איטרציה, הסביבה saved-env מורחבת במשתנה אחד מהרשימה ids וערכו נקבע או מ-args או מ- defs בהתאם להאם הוא קיים ב- params לפי האינדקס שבו הוא נוכח.

פתרון שאלה 2

להלן השינויים בקובץ lang.scm :

(expression

(type "[" expression "]" "{" expression (arbno " " expression) "}")

arr-exp)

(type

("#")

type-pound)

סה"כ פתרון טוב ונכון, למעט שגיאות קלות

(type

("?")

type-question)

(type

("@")

type-at)

(expression

("[" expression " " expression "]")

index-exp)

השינויים ב- interp.scm בתוך value-of (הוספת case) הם:

(arr-exp (typ size first-exp more-exps)

(let (

(vals (map (lambda (exp) (value-of exp env)) (cons first-exp more-exps)))

(size (expval->num (value-of size env)));size of the array

)

(if (= (length vals) size)

(array-val typ size (helper typ vals '())) ; NEW DATA TYPE DEFINED IN datastructure.scm

) (eopl:error "size doesn't match"))

)

זה עדיין ביטוי שלא חושב. יש לחשבו תחילה

```
)
(eopl:error "must be an array")
)
)
```

```
(define (helper typ vals lst)
  (cases type typ
    (type-question () )
    (type-pound ()
      (if (null? vals) lst
          (cases expval (car vals)
            (numval (num) (helper (cdr vals) (cons (newref (car vals)) lst)))
            (else (eopl:error "type mismatch"))
          )
        )
    (type-at () )
  ))
```

?



בתוך data-structure.scm יש להוסיף תחת `define-datatype expval expval` ?בנאי חדש:

```
(array-val (typ type?) (size integer?) (arr (list-of integer?)))
```

הסיבה לכך שהמערך הוא רשימה של שלמים היא שכתובת בזיכרון הם כאילו.

כאשר מחזירים reference, כלומר כתובת ב-store, אין אכיפה לגבי טיפוס הנתונים typ. אנו נותנים גישה ישירה לזיכרון.

לא הוספתי מחלץ לבנאי הזה כמקובל וזאת מכיוון שממילא אני מחלץ באמצעות cases כי יש כמה רכיבים לבנאי זה.

הערה חשובה: במקום ש-arr יהיה רשימה של integer, היה יותר נכון שיהיה רשימה של ref-val ואז בהתאם שאר הפרוצדורות צריכות להשתנות. ואז באמצעות המחלץ `expval->ref` נקבל את ref שהוא הכתובת.

בדיוק זה מה
שצריך על מנת
לשמור על תאימות
עם שאר ביטויים
בשפה

פתרון שאלה 3

הגדרת שמות משתנים לטיפוסים:



let p1 = proc(x:?) (x 9) in let p2=proc(y:?) zero?(y) in (if (p1 p2) then p2 else proc(z:?) zero?(-(z,5)) 7)	t_0
p1	t_{p1}
proc(x:?) (x 9)	t_1
let p2=proc(y:?) zero?(y) in (if (p1 p2) then p2 else proc(z:?) zero?(-(z,5)) 7)	t_2
p2	t_{p2}
proc(y:?) zero?(y)	t_3
(if (p1 p2) then p2 else proc(z:?) zero?(-(z,5)) 7)	t_4
x	t_x
(x 9)	t_5
y	t_y
zero?(y)	t_6
if (p1 p2) then p2 else proc(z:?) zero?(-(z,5))	t_7
(p1 p2)	t_8
proc(z:?) zero?(-(z,5))	t_9
z	t_z
zero?(-(z,5))	t_{10}
-(z,5)	t_{11}

בניית משוואות:

let p1 = proc(x:?) (x 9) in let p2=proc(y:?) zero?(y) in (if (p1 p2) then p2 else proc(z:?) zero?(-(z,5)) 7)	$t_0 = t_2$ $t_{p1} = t_1$
proc(x:?) (x 9)	$t_1 = (t_x \rightarrow t_5)$
let p2=proc(y:?) zero?(y) in (if (p1 p2) then p2 else proc(z:?) zero?(-(z,5)) 7)	$t_{p2} = t_3$ $t_2 = t_4$
proc(y:?) zero?(y)	$t_3 = (t_y \rightarrow t_6)$
(if (p1 p2) then p2 else proc(z:?) zero?(-(z,5)) 7)	$t_7 = (int \rightarrow t_4)$
(x 9)	$t_x = (int \rightarrow t_5)$
zero?(y)	$t_y = int$ $t_6 = bool$
if (p1 p2) then p2 else proc(z:?) zero?(-(z,5))	$t_8 = bool$ $t_{p2} = t_7$ $t_9 = t_7$
(p1 p2)	$t_{p1} = (t_{p2} \rightarrow t_8)$
proc(z:?) zero?(-(z,5))	$t_9 = (t_z \rightarrow t_{10})$

zero?(-(z,5))	$t_{10} = bool$
	$t_{11} = int$
-(z,5)	$t_{11} = int$
	$t_z = int$

פתרון המשוואות עפ"י האלגוריתם:

$t_0 = t_2$	
$t_{p1} = t_1$	
$t_1 = (t_x \rightarrow t_5)$	
$t_{p2} = t_3$	
$t_2 = t_4$	
$t_3 = (t_y \rightarrow t_6)$	
$t_7 = (int \rightarrow t_4)$	
$t_x = (int \rightarrow t_5)$	
$t_y = int$	
$t_6 = bool$	
$t_8 = bool$	
$t_{p2} = t_7$	
$t_9 = t_7$	
$t_{p1} = (t_{p2} \rightarrow t_8)$	
$t_9 = (t_z \rightarrow t_{10})$	
$t_{10} = bool$	
$t_{11} = int$	
$t_{11} = int$	
$t_z = int$	

(אני מרשה לעצמי לעשות כמה שלבים ביחד בעבור המקרים הטריטיויאליים)



	$t_0 = t_2$
	$t_{p1} = t_1$
$t_1 = (t_x \rightarrow t_5)$	
$t_{p2} = t_3$	
$t_2 = t_4$	
$t_3 = (t_y \rightarrow t_6)$	
$t_7 = (int \rightarrow t_4)$	
$t_x = (int \rightarrow t_5)$	

$t_y = int$	
$t_6 = bool$	
$t_8 = bool$	
$t_{p2} = t_7$	
$t_9 = t_7$	
$t_{p1} = (t_{p2} \rightarrow t_8)$	
$t_9 = (t_z \rightarrow t_{10})$	
$t_{10} = bool$	
$t_{11} = int$	
$t_{11} = int$	
$t_z = int$	



	$t_0 = t_4$
	$t_{p1} = ((int \rightarrow t_5) \rightarrow t_5)$
	$t_1 = ((int \rightarrow t_5) \rightarrow t_5)$
	$t_{p2} = (t_y \rightarrow t_6)$
	$t_2 = t_4$
	$t_3 = (t_y \rightarrow t_6)$
	$t_7 = (int \rightarrow t_4)$
	$t_x = (int \rightarrow t_5)$
$t_y = int$	
$t_6 = bool$	
$t_8 = bool$	
$t_{p2} = t_7$	
$t_9 = t_7$	
$t_{p1} = (t_{p2} \rightarrow t_8)$	
$t_9 = (t_z \rightarrow t_{10})$	
$t_{10} = bool$	
$t_{11} = int$	
$t_{11} = int$	
$t_z = int$	

	$t_0 = t_4$
	$t_{p1} = ((int \rightarrow t_5) \rightarrow t_5)$
	$t_1 = ((int \rightarrow t_5) \rightarrow t_5)$
	$t_{p2} = (int \rightarrow bool)$
	$t_2 = t_4$
	$t_3 = (int \rightarrow bool)$
	$t_7 = (int \rightarrow t_4)$
	$t_x = (int \rightarrow t_5)$
	$t_y = int$
	$t_6 = bool$
$t_8 = bool$	
$t_{p2} = t_7$	
$t_9 = t_7$	
$t_{p1} = (t_{p2} \rightarrow t_8)$	
$t_9 = (t_z \rightarrow t_{10})$	
$t_{10} = bool$	
$t_{11} = int$	
$t_{11} = int$	
$t_z = int$	

	$t_0 = bool$
	$t_{p1} = ((int \rightarrow t_5) \rightarrow t_5)$
	$t_1 = ((int \rightarrow t_5) \rightarrow t_5)$
	$t_{p2} = (int \rightarrow bool)$
	$t_2 = bool$
	$t_3 = (int \rightarrow bool)$
	$t_7 = (int \rightarrow bool)$
	$t_x = (int \rightarrow t_5)$
	$t_y = int$
	$t_6 = bool$
	$t_8 = bool$
	$t_{p2} = (int \rightarrow t_4)$
$t_9 = t_7$	$(int \rightarrow t_4) = (int \rightarrow bool) \Rightarrow t_4 = bool$
$t_{p1} = (t_{p2} \rightarrow t_8)$	
$t_9 = (t_z \rightarrow t_{10})$	



$t_{10} = bool$	
$t_{11} = int$	
$t_{11} = int$	
$t_z = int$	



משלב זה אנחנו כבר יודעים שהתוצאה האפשרית היחידה היא bool ורק נותר לוודא שאין סתירה.

	$t_0 = bool$
	$t_{p1} = ((int \rightarrow t_5) \rightarrow t_5)$
	$t_1 = ((int \rightarrow t_5) \rightarrow t_5)$
	$t_{p2} = (int \rightarrow bool)$
	$t_2 = bool$
	$t_3 = (int \rightarrow bool)$
	$t_7 = (int \rightarrow bool)$
	$t_x = (int \rightarrow t_5)$
	$t_y = int$
	$t_6 = bool$
	$t_8 = bool$
	$t_{p2} = (int \rightarrow t_4)$
	$(int \rightarrow t_4) = (int \rightarrow bool) \Rightarrow t_4 = bool$
	$t_9 = (int \rightarrow bool)$
$t_9 = (t_z \rightarrow t_{10})$	$t_{p1} = ((int \rightarrow bool) \rightarrow bool)$
$t_{10} = bool$	
$t_{11} = int$	
$t_{11} = int$	
$t_z = int$	



	$t_0 = bool$
	$t_{p1} = ((int \rightarrow t_5) \rightarrow t_5)$
	$t_1 = ((int \rightarrow t_5) \rightarrow t_5)$
	$t_{p2} = (int \rightarrow bool)$
	$t_2 = bool$
	$t_3 = (int \rightarrow bool)$
	$t_7 = (int \rightarrow bool)$
	$t_x = (int \rightarrow t_5)$

	$t_y = int$
	$t_6 = bool$
	$t_8 = bool$
	$t_{p2} = (int \rightarrow t_4)$
	$(int \rightarrow t_4) = (int \rightarrow bool) \Rightarrow t_4 = bool$
	$t_9 = (int \rightarrow bool)$
	$t_{p1} = ((int \rightarrow bool) \rightarrow bool)$
	$t_9 = (t_z \rightarrow bool)$
$t_{11} = int$	$t_{10} = bool$
$t_{11} = int$	
$t_z = int$	



ומכאן העסק כבר קל:

	$t_0 = bool$
	$t_{p1} = ((int \rightarrow bool) \rightarrow bool)$
	$t_1 = ((int \rightarrow bool) \rightarrow bool)$
	$t_{p2} = (int \rightarrow bool)$
	$t_2 = bool$
	$t_3 = (int \rightarrow bool)$
	$t_7 = (int \rightarrow bool)$
	$t_x = (int \rightarrow bool)$
	$t_y = int$
	$t_6 = bool$
	$t_8 = bool$
	$t_{p2} = (int \rightarrow bool)$
	$(int \rightarrow t_4) = (int \rightarrow bool) \Rightarrow t_4 = bool$
	$t_9 = (int \rightarrow bool)$
	$t_{p1} = ((int \rightarrow bool) \rightarrow bool)$
	$t_9 = (int \rightarrow bool)$
	$t_{10} = bool$
	$t_{11} = int$
	$t_z = int$
	$((int \rightarrow t_5) \rightarrow t_5) = ((int \rightarrow bool) \rightarrow bool) \Rightarrow t_5 = bool$

ולכן ניתן להקיש שהטיפוס של התוכנית הוא **bool**.



